

Devoir surveillé terminal

Durée 2h
(Corrections)

*Aucun document, outil de calcul ou de communication autorisé. Sauf indications contraires, la gestion d'erreur **ne doit pas** être réalisée et les inclusions ne doivent pas être spécifiées.
Toute réponse doit être justifiée.*

Bien qu'indépendants, les exercices de ce sujet suivent la modélisation suivante.

Nous souhaitons développer une simulation d'arène sur laquelle combattent des robots. L'arène est découpée en cases : sur chacune, il ne peut y avoir qu'un robot, un obstacle ou rien. Cette simulation correspond en fait à trois applications : le *synchroniseur*, le *serveur* et les *clients* (correspondant aux robots).

La simulation est découpée en tours pendant lesquels chaque robot ne peut faire qu'une action : attaquer un robot situé sur une case adjacente (haut, bas, droite et gauche), attendre ou se déplacer (sur une case vide). Si l'action choisie n'est pas possible (si un robot s'est placé sur la case vide sur laquelle le robot voulait se déplacer ou si le robot attaqué s'est déplacé entre temps), le robot perd alors son action et reste sur sa position.

Le synchroniseur crée l'arène puis met en place les différents outils IPC dont les clés sont fixées par les constantes `CLE_SEGMENT`, `CLE_FILE` et `CLE_SEMAPHORE` puis les initialise. Il affiche ensuite l'état de la simulation à l'aide d'une interface *ncurses* : une première fenêtre permet d'afficher les logs (lorsqu'un robot se connecte, est détruit ou réalise une action) et l'état de l'arène est affiché dans une deuxième fenêtre. C'est le synchroniseur qui est en charge de gérer les tours. Il prend en arguments les dimensions de l'arène (largeur et hauteur) et le nombre maximum de robots.

Le serveur prend en argument un numéro de port d'écoute TCP. Lorsqu'un client se connecte, le serveur crée un processus fils qui est en charge de la connexion établie. Il envoie ensuite un message via la file de messages au synchroniseur pour l'avertir de l'arrivée du robot. Le processus fils communique ensuite avec le client : envoi de l'état de l'arène, demande d'ordres, attaques éventuelles et réception des actions du robot. Il envoie les actions au synchroniseur via la file de messages une fois l'arène mise à jour.

Le client correspond à un patron permettant ainsi de simuler des robots avec des comportements différents et peut être exécuté plusieurs fois. C'est une application *ncurses* qui prend en arguments l'adresse IP du serveur et son numéro de port.

Exercice 1 (Le segment de mémoire partagée (4 points))

Le segment de mémoire partagée contient toutes les données accessibles à la fois par le synchroniseur, le serveur et ses processus fils. Il contient les dimensions de l'arène, l'arène en elle-même (les cases), le nombre maximum de robots et les points de vie de chacun (à 10 au départ, décrémenté de 1 à chaque attaque subie).

1°) Précisez le contenu du segment de mémoire partagée (données + types) puis proposez une structure `segment_t` permettant de le mapper tel que vu en CM/TD (elle contiendra notamment un champ pour l'identifiant du segment et un pour son adresse mémoire).

Solution : (1 point) *Le segment contient les données suivantes :*

- Largeur, hauteur : *unsigned char*
- Cases : *unsigned char* $\times$ largeur $\times$ hauteur
- Nombre max. de clients : *unsigned char*

— Points de vie : `unsigned char` $\times$ nombre max. de clients

Une structure pour mapper le segment :

```
typedef struct {
    unsigned char *debut;
    int id;
    unsigned char *largeur, *hauteur, *cases, *maxClients, *vieClients;
} segment_t;
```

2°) Donnez le code de la fonction `recup_segment` permettant à un fils du serveur de récupérer le contenu du segment de mémoire partagée. Elle ne prend aucun paramètre et retourne un pointeur sur un `segment_t` qui permet de mapper le segment.

Solution : (3 points) Voici une solution :

```
segment_t *recup_segment() {
    segment_t *resultat = (segment_t*) malloc(sizeof(segment_t));
    resultat->id = shmget(CLE_SEGMENT, 0, 0);
    resultat->debut = shmat(resultat->id, NULL, 0);

    resultat->largeur = &debut[0];
    resultat->hauteur = &debut[1];
    resultat->cases = &debut[2];
    resultat->maxClients = &debut[2 + *resultat->largeur * *resultat->hauteur];
    resultat->vieClients = &resultat->vieClients[1];

    return resultat;
}
```

Exercice 2 (Les problèmes de synchronisation (7 points))

1°) Nous souhaitons que les fils du serveur attendent passivement que le tour commence : c'est le synchroniseur qui le déclenche. Proposez une solution en utilisant les sémaphores (décrivez les sémaphores utilisés, leur initialisation, ainsi que les opérations nécessaires).

Solution : (1 point) Il est possible d'utiliser un sémaphore S_1 , initialisé à n avec n le nombre de robots actuellement connecté. Chaque processus fils fait alors un $P(S_1, 1)$ sur ce sémaphore. Lorsque le tour commence, le synchroniseur fait un $V(S_1, n)$.

2°) Que se passe-t-il si un robot se connecte entre temps? Votre solution fonctionne-t-elle? Expliquez et proposez une solution si ce n'est pas le cas (toujours à l'aide de sémaphores).

Solution : (1 point) Si un robot se connecte entre temps, il fera un $P(S_1, 1)$ qui risque de passer avant un robot déjà connecté. Il suffit donc de bloquer le robot tant que le synchroniseur n'a pas pris en compte sa demande. Une solution est d'utiliser un deuxième sémaphore initialisé à 0. Lorsqu'un robot se connecte, il fait un $V(S_2, 1)$ puis un $ATT(S_2)$. Dès que le synchroniseur prend en compte son arrivée, il fait un $P(S_2, 1)$ et incrémente son nombre de robots.

3°) Expliquez les problèmes de synchronisation qui peuvent survenir lors des actions des robots. Proposez une solution à base de sémaphores pour les résoudre en limitant leur nombre sans empêcher les exécutions parallèles.

Solution : (2 points) Pour se déplacer, il faut que le robot bloque sa propre case puis la case sur laquelle il souhaite aller. Cependant, si ce n'est pas possible, cela signifie qu'un autre robot est situé sur cette case, par exemple, et qu'il bloque déjà la case. Une solution : la deuxième opération est non bloquante et si elle échoue, le robot perd son tour.

Cette solution nécessite 1 sémaphore par case. Une solution pour réduire le nombre de sémaphores, c'est de bloquer l'arène par zones (en découpant la grille en sous-grilles de 3 par 3, par exemple). Attention cependant, car cela risque d'impliquer de bloquer plusieurs zones lors d'un déplacement d'un robot d'une zone à l'autre.

4°) Donnez le code permettant au synchroniseur de créer le tableau de sémaphores en fonction de vos réponses aux questions précédentes et de l'initialiser. Si le tableau existe, il est supprimé, sauf si le nombre de sémaphores correspond (dans ce cas, on initialise simplement les sémaphores).

Solution : (3 points) Voici une solution :

```
int semid = semget(CLE_SEMAPHORE, n, S_IRUSR | S_IWUSR | IPC_CREAT | IPC_EXCL);
if((semid == - 1) && (errno == EEXIST)) {
    semid = semget(CLE_SEMAPHORE, 0, S_IRUSR | S_IWUSR);
    struct semid_ds infos;
    semctl(semid, 0, IPC_STAT, &infos);
    if(infos.sem_nsems != n) {
        semctl(semid, 0, IPC_RMID);
        semid = semget(CLE_SEMAPHORE, n, S_IRUSR | S_IWUSR | IPC_CREAT | IPC_EXCL);
    }
}
unsigned short tableau[n];
/* Initialisation du tableau */
semctl(semid, 0, SETALL, tableau);
```

Exercice 3 (Le serveur (5 points))

Le serveur récupère le nombre de robots maximum via le segment de mémoire partagée. Lorsqu'une demande de connexion est reçue, le serveur vérifie si le nombre maximum de robots est déjà atteint. Si c'est le cas, il envoie un code d'erreur (un **unsigned char**) au client. Sinon, il envoie un code de succès (un **unsigned int**) puis crée un processus fils.

1°) Sachant qu'un robot peut s'arrêter à tout moment, proposez une solution permettant au serveur de connaître le nombre exact de clients connectés.

Solution : (1 point) Le serveur possède une variable permet de compter le nombre de connexions, initialisée à 0. Dès qu'une connexion est établie, le serveur compare le nombre de connexions avec ce nombre. Si c'est OK, il crée un fils et incrémente la variable. Lorsqu'un fils s'arrête, le père peut le détecter à l'aide du signal **SIGCHLD** qui est reçu à chaque fin de processus fils. En positionnant un gestionnaire sur ce signal et en utilisant une variable globale, dès que le signal est reçu, on décrémente la variable. Attention cependant à **accept** qui sera interrompu et retournera -1.

2°) Donnez la portion de code du serveur qui attend une connexion (en supposant la socket déjà créée et nommée), envoie le code de succès/d'erreur au client (en fonction du nombre actuel de clients) puis crée le fils associé (si le nombre maximum de clients n'est pas atteint).

- La procédure **fils** correspond au code d'un fils : vous donnerez sa signature en expliquant chaque paramètre.
- Chaque variable devra être déclarée (et initialisée si besoin).
- La fonction **getNumero()** retourne le numéro du nouveau client (entre 0 et le nombre maximum de clients moins 1) ou -1 si le nombre maximum de clients est atteint.

Solution : (3 points) Voici une solution :

```
/**
 * fd : descripteur de fichier de la socket (communication)
 * numClient : le numéro du client
 */
void fils(int fd, int numClient);

/* socket+bind */
int numClient;
unsigned char code;

int commfd = accept(sockfd, NULL, 0);
if((numClient = getNumero()) != -1) {
    code = CODE_OK;
```

```

    write(commfd, &code, sizeof(int));
    if(fork() == 0) {
        close(sockfd);
        fils(commfd, numClient);
    }
    else {
        close(commfd);
    }
}
else {
    code = CODE_KO;
    write(commfd, &code, sizeof(int));
    close(commfd);
}
}

```

3°) Donnez toutes les modifications de code (variables nécessaires comprises) pour afficher sur le terminal l'adresse du client (IP+port) qui a demandé une connexion.

Solution : (1 point) Voici les modifications :

```

struct sockaddr_in adr;
socklen_t socklen;
char adresseIP20[];

...
memset(&adr, 0, socklen);
socklen = sizeof(adr);
int commfd = accept(sockfd, (struct sockaddr*)&adr, &socklen);
inet_ntop();
printf("Client_%s:d\n", adresseIP, ntohs(adr.sin_port));

```

Exercice 4 (Les communications avec le synchroniseur (4 points))

Lorsque des robots se déplacent, les fils du serveur mettent à jour directement l'arène dans le segment de mémoire partagée (mise-à-jour des cases ou des points de vie des robots). Pour qu'il soit averti de ces actions, des messages sont envoyés au synchroniseur soit par le serveur (lors des connexions), soit par les fils du serveur.

1°) Proposez une/des structures représentant les messages envoyés dans la file de messages.

Solution : (1 point) Voici une solution :

```

#define TYPE_CONNEXION 1
#define TYPE_DECONNEXION 2
#define TYPE_DEPLACEMENT 3
#define TYPE_ATAQUE 4
#define TYPE_ATTENTE 5

typedef struct { int num; } connexion_t;
typedef struct { int num; } deconnexion_t;
typedef struct { int num; int x, y; } deplacement_t;
typedef struct { int num; int cible; } attaque_t;
typedef struct { int num; } attente_t;

typedef struct {
    long type;
    union_t {
        connexion_t c;
        deconnexion_t d;
        ...
    } donnees;
}

```

```
} message_t;
```

2°) Donnez les deux portions de code du serveur permettant de récupérer la file et d'envoyer un message pour avertir le synchroniseur de la connexion d'un nouveau client.

Solution : (2 points) Voici une solution :

```
int msgid = msgget(CLE_FILE, S_IRUSR | S_IWUSR);  
  
message_t msg = { ... };  
msgsnd(msgid, &msg, sizeof(message_t) - sizeof(long), 0);
```

3°) Rappelez les problèmes rencontrés avec une file pour la récupération de messages de longueurs différentes. Proposez une solution.

Solution : (1 point) Lorsqu'un message est lu, il est supprimé de la file. Or, la taille n'est connue qu'une fois le message lu. Une solution : utiliser des messages de taille fixe (avec une union, par exemple) ou bien utiliser les types de messages (lire un type donné, puis un autre, etc.) ce qui implique une attente active.